



Université Abdelmalek Essaâdi · Ecole Nationale des Sciences
Appliquées d'Al Hoceima



AFNDetection

Arabic Fake News Detection Using a Fine-Tuned AraBERT Transformer

A deep-learning system and deployable web application for classifying Modern Standard Arabic news



AUTHOR Zaynab AITADDI

INSTRUCTOR Mr. Cherradi

COURSE Deep Learning

DATE May 17, 2026

Abstract

Misinformation spreads quickly on Arabic digital platforms, but most fake news detection research focuses on English. This project presents **AFNDEtection**, a system that uses **AraBERT v2** to classify Arabic news as **real or fake**. Due to the lack of labeled Arabic data, a dataset of **5,293 samples** was created using curated data, public datasets, and data augmentation. After fine-tuning, the model achieved **83% accuracy** and an **F1-score of 82.9%** on a balanced test set. The model was deployed in a **Flask web application** with a right-to-left Arabic interface for real-time prediction. The entire system is reproducible and can support future Arabic NLP research.

Keywords: *Arabic NLP, fake news detection, AraBERT, transformers, fine-tuning, data augmentation, label smoothing, layer-wise learning-rate decay, Flask, right-to-left interface.*

Table of Contents

Abstract 2

Table of Contents 2

1 Introduction 4

 1.1 Context 4

 1.2 Problem Statement 4

 1.3 Contributions 4

 1.4 Report Structure 5

2 Literature Review 5

 2.1 Arabic NLP and Transformer Models 5

 2.2 Fake News Detection Approaches 5

 2.3 Existing Arabic Fake News Datasets 6

 2.4 Positioning of This Work 6

3 Methodology 6

 3.1 Dataset Description 7

 3.1.1 Primary Data Source 7

 3.1.2 Augmented and External Data 7

 3.1.3 Text Composition 8

 3.1.4 Sample Examples 8

 3.2 Data Preprocessing 8

 3.2.1 Train / Validation / Test Split 8

 3.3 Model Architecture 9

 3.3.1 Base Model: AraBERT v2 9

 3.3.2 Classification Head 10

 3.4 Training Configuration 10

 3.4.1 Hyperparameters 10

3.4.2 Optimization Strategy	11
3.4.3 Evaluation Metrics.....	11
3.5 Training Infrastructure.....	12
3.6 Model Checkpoint Export	12
3.7 Web Application Architecture	13
3.7.1 Backend (Flask).....	13
3.7.2 Frontend (HTML / CSS / JS)	13
3.7.3 Configuration.....	15
4 Results and Discussion.....	15
4.1 Training Results.....	15
4.2 Test Set Evaluation	16
4.3 Inference Examples.....	17
4.4 Interpretation	18
4.5 Comparison with Literature	18
5 Limitations and Future Work.....	18
5.1 Limitations	18
5.2 Ethical Considerations	19
5.3 Future Work.....	19
6 Conclusion	20
References	20
Appendices.....	21
A Project File Structure	21
B Training Commands	22
C Web Application Launch.....	22
D Test Metrics	22
E Hardware and Environment Specifications.....	22

1 Introduction

1.1 Context

Misinformation is not a new phenomenon, but the velocity at which it now propagates is unprecedented. Arabic-speaking regions across North Africa and the Middle East are particularly exposed: social-media penetration is high while fact-checking infrastructure remains uneven. Fabricated content circulates through WhatsApp groups, Facebook pages, and X (formerly Twitter) faster than it can be verified.

Manual fact-checking cannot operate at this scale. Verifying the volume of Arabic content published daily would require a continuous effort by large teams of trained journalists. Automated detection is therefore not a convenience but a practical necessity.

1.2 Problem Statement

The overwhelming majority of mature fake-news detection research targets English. Arabic is comparatively underserved, despite presenting several properties that make it intrinsically harder to model:

- **Rich morphology.** A single Arabic root generates dozens of surface forms through prefixes, suffixes, infixes, and internal vowel changes. Where English derives a small family such as write / writer / wrote / written, Arabic produces a much larger and more entangled set of inflections.
- **Diglossia.** Modern Standard Arabic (MSA), used in formal news, coexists with Egyptian, Levantine, Gulf, and Maghrebi dialects that differ substantially. This work addresses MSA; dialectal social-media text is a distinct and harder problem.
- **Right-to-left script.** Most web frameworks assume left-to-right layout, so building a faithful Arabic interface requires deliberately overriding CSS direction, alignment, and cursor behaviour.
- **Scarce labelled data.** English benefits from established corpora such as LIAR, FakeNewsNet, and ISOT. Arabic offers only a handful of datasets, most of which are small or noisy.
- **Subtle deception cues.** Fabricated Arabic news exhibits characteristic markers, heavy use of the urgency term عاجل (“breaking”), unsourced miracle-cure claims, and conspiratorial framing that asserts a hidden truth. General-purpose Arabic models do not reliably capture these patterns without domain-specific fine-tuning.

1.3 Contributions

This project delivers the following:

- **End-to-end training pipeline.** A complete workflow covering data merging, augmentation, layer-wise learning-rate decay, label smoothing, early stopping, and checkpoint export, runnable both on Google Colab and locally.
- **Curated seed dataset.** A corpus of 170 texts cleaned to 100 high-quality balanced samples, demonstrating that careful pipeline design can offset limited manual annotation.

- **Scalable data strategy.** The seed corpus, public Hub data, and controlled augmentation combine to 5,293 training samples, illustrating one route to amplifying scarce labelled resources.
- **Deployable web application.** A Flask backend with an Arabic right-to-left frontend providing real-time inference and probability visualisation, beyond a research prototype.
- **Reproducible configuration.** Versioned YAML files specify every component, allowing independent replication of the exact setup.
- **Accessible training workflow.** A Colab notebook with automatic Google Drive synchronisation enables GPU training without dedicated hardware, lowering the barrier for students and resource-constrained researchers.

1.4 Report Structure

Section 2 reviews related work. Section 3 details the methodology, from data to deployment. Section 4 presents the quantitative results and their interpretation. Section 5 discusses limitations and future directions, and Section 6 concludes.

2 Literature Review

2.1 Arabic NLP and Transformer Models

Arabic natural-language processing historically trailed English, constrained by rule-based systems and classical machine learning with hand-crafted features. Transformer architectures changed this trajectory.

- **AraBERT (Antoun et al., 2020).** The model adopted in this work. Pre-trained on roughly 70 million Arabic sentences (8.5 billion tokens) drawn from news, Wikipedia, and books, with Arabic-specific tokenisation that handles morphology better than generic byte-pair encoding.
- **MARBERT (Abdul-Mageed et al., 2021).** Trained on one billion Arabic tweets and strong on dialectal and social-media text, but heavier and unnecessary for formal news.
- **ARBERT.** Trained on a larger, more diverse web corpus; capable, though its additional scale was not required here.

AraBERT v2 was selected because news-style MSA matches its pre-training distribution, its vocabulary captures Arabic morphological patterns that multilingual BERT misses, and at 110 million parameters it fits comfortably within a 16 GB T4 GPU, an appropriate balance of accuracy and resource cost for a student or small-team setting.

2.2 Fake News Detection Approaches

Three broad strategies dominate the literature. This work adopts the content-based approach, since it requires no social-graph data and operates on a single article.

Approach	Mechanism	Strengths	Weaknesses
Content-based	Analyses the text itself: style, sentiment, word choice, semantic patterns	No external data required; works on a single article	Language-dependent; adaptable authors can evade it
Propagation-based	Tracks diffusion: who shares, when, and the network structure	Captures viral misinformation; exploits social context	Requires network data; costly; raises privacy concerns
Hybrid	Combines content and propagation signals	More robust to attacks on a single modality	Complex; difficult to align all data sources

The underlying hypothesis is that deceptive Arabic news carries learnable linguistic fingerprints: the urgency marker عاجل, pseudoscientific claims lacking verifiable sources, and emotionally loaded language are all detectable given sufficient examples and a model with genuine Arabic context.

2.3 Existing Arabic Fake News Datasets

- **Nahla-yasmine/arabic_fake_news (Hugging Face).** Approximately 200k web-scraped samples; large but of variable quality. Up to 5,000 balanced samples were used from this source.
- **ArFake-2021.** Cleaner and politically focused; not used directly but drawn from a similar domain.
- **AraNews.** General news classification; adaptable but not labelled for the fake/real distinction.

The strategy combines a small curated seed (100 guaranteed-quality samples), a larger Hub corpus for quantity, and augmentation for variability. The deliberately small seed demonstrates that, with sound pipeline design, limited high-quality labels can be stretched a long way.

2.4 Positioning of This Work

Many studies stop at reporting a benchmark score on a single dataset. This project targets three specific gaps:

- **Research-to-deployment gap.** Benchmark scores are useful, but a functioning web interface is more so.
- **English-centric gap.** The interface is Arabic-native rather than a translation: right-to-left layout, Arabic labels, and Arabic error messages.
- **Reproducibility gap.** YAML configurations, pinned versions, and Colab notebooks with Drive synchronisation make independent replication straightforward.

3 Methodology

This section describes the system end to end, from data acquisition through model training to web deployment. Figure 1 gives the overall architecture: a data layer feeding a training layer, whose exported checkpoint serves a deployment layer.

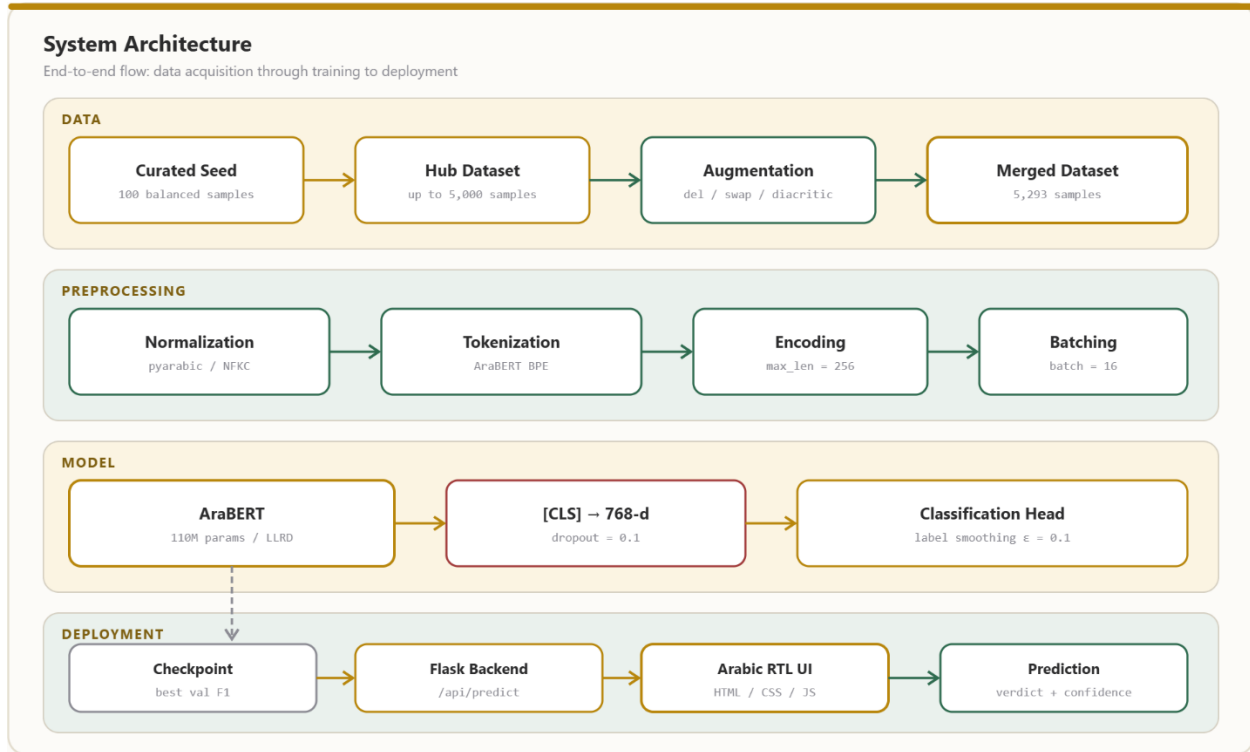


Figure 1. System architecture, from the data layer through training to web deployment.

3.1 Dataset Description

3.1.1 Primary Data Source

The curated file `data/raw/arabic_curated.csv` initially contained 170 rows. After discarding entries with an empty title or body, 100 balanced samples remained, 50 real and 50 fake, forming the seed corpus.

Class	Count	Typical content
Real (1)	50	Government announcements, weather reports, sports results, economic updates, research summaries
Fake (0)	50	Miracle cures, conspiracy theories, fabricated celebrity deaths, pseudoscience, urgent viral claims

3.1.2 Augmented and External Data

Two strategies expanded the corpus beyond the 100 seed samples. A Hub dataset contributed up to 5,000 balanced samples from `Nahla-yasmine/arabic_fake_news`. Offline augmentation then produced two variants of each curated sample through random word deletion (15% of words), two random adjacent-word swaps, and diacritic-noise insertion (5% of characters). The final training set contained 5,293 roughly balanced samples. Figure 2 summarises the class balance, the stratified splits, and the normalisation pipeline.

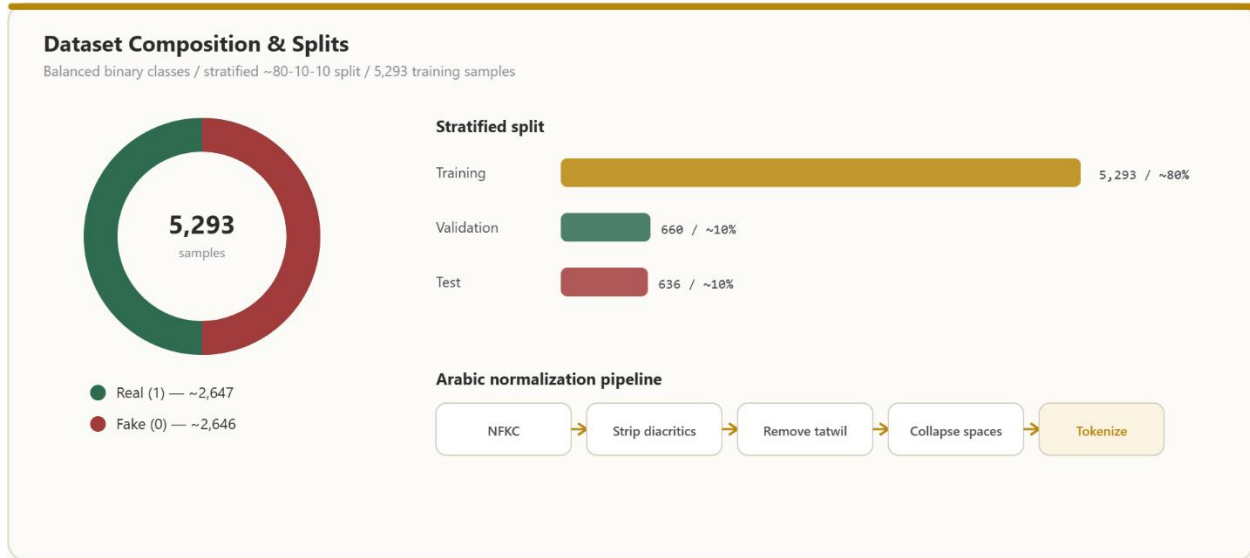


Figure 2. Dataset composition: class balance, stratified splits, and the normalisation pipeline.

3.1.3 Text Composition

Each sample comprises a title (10–20 words), a body (50–150 words), and a binary label (0 or 1). During training, the title and body are concatenated (title_plus_text mode) so the model sees both the headline context and the body detail.

3.1.4 Sample Examples

Real (label 1) — title: “وزارة التربية تعلن موعد امتحانات الدورة الربيعية لجميع الأسلاك”; body: “أوضحت الوزارة في بلاغ أن الامتحانات ستطلق وفق الجدولة المعتمدة”.

Fake (label 0) — title: “أطباء في دراسة سرية يكشفون: هذا العشب يقضي على السكري في ثمان وأربعين ساعة فقط”; body: advances claims without statistics, omits hospital names, and directs the reader to suspicious links.

3.2 Data Preprocessing

Arabic text is orthographically noisy: a single character may have several Unicode representations, diacritics (tashkeel) appear inconsistently, and decorative kashida elongation confuses tokenisers. The following normalisation steps were applied with the lightweight pyarabic library.

Operation	Purpose	Example
Unicode NFKC normalisation	Standardise character forms	ا ← ا
Diacritic removal	Reduce sparsity and normalise	السَّلام ← السلام
Tatwil (kashida) removal	Remove elongation artifacts	مرحبا ← مرحبا
Whitespace normalisation	Clean formatting	Multiple spaces → single space

Without normalisation, the tokeniser would treat distinct presentation forms as different tokens: an isolated ا and a final-form ا would occupy separate vocabulary entries, fragmenting the model’s understanding of what is essentially the same letter and wasting representational capacity.

3.2.1 Train / Validation / Test Split

Splitting was stratified, preserving the 50/50 class balance in every partition (val_ratio = 0.10, test_ratio = 0.10, seed = 42).

Split	Ratio	Real	Fake	Total
Training	80%	~2,647	~2,647	5,293
Validation	10%	~330	~330	~660
Test	10%	318	318	636

Stratification matters because random splitting on small data can yield a validation set with no fake examples, rendering its metrics meaningless. Stratification guarantees both classes appear in every partition.

3.3 Model Architecture

Figure 3 illustrates the AraBERT encoder together with the classification head.

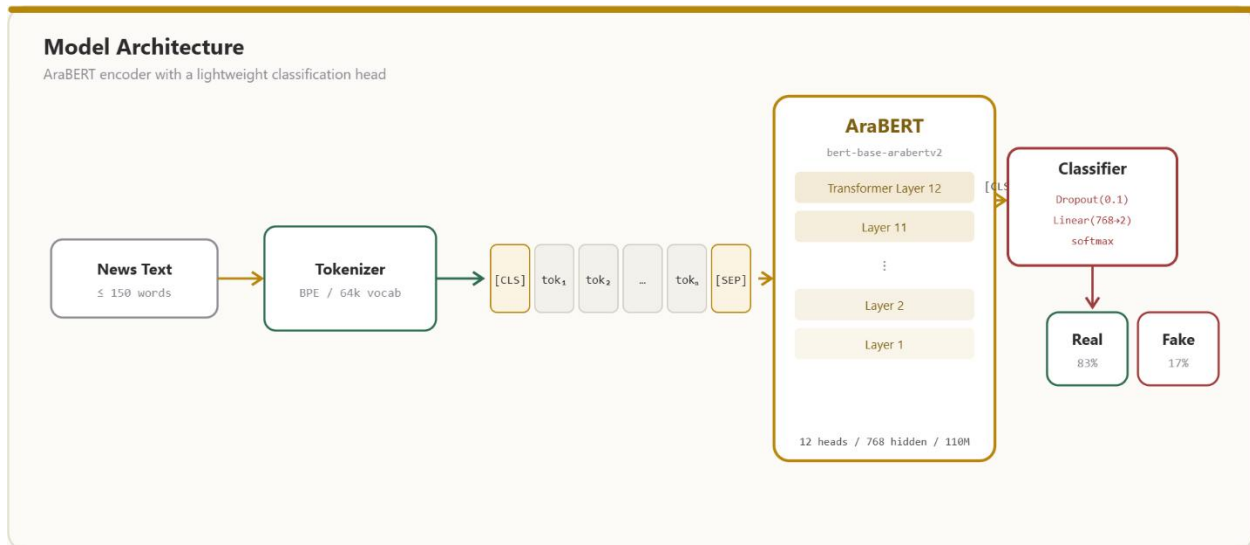


Figure 3. AraBERT v2 encoder with the classification head.

3.3.1 Base Model: AraBERT v2

Property	Value
Model ID	aubmindlab/bert-base-arabertv2
Architecture	BERT-base, 12 transformer layers
Hidden size	768
Attention heads	12
Parameters	110 million
Pre-training corpus	70M Arabic sentences (8.5B tokens)
Max sequence length	256 tokens
Vocabulary size	~64k subword tokens (Arabic-optimised BPE)

BERT was preferred over recurrent architectures for three reasons. Bidirectional attention reads context in both directions simultaneously, which matters in Arabic where negation particles often begin long sentences and govern words far to their right. Transfer learning from 8.5 billion tokens supplies syntax, morphology, and semantics that fine-tuning then adapts to deception detection. Finally, subword tokenisation handles out-of-vocabulary words gracefully by composing them from shared units.

3.3.2 Classification Head

The pipeline is: the 768-dimensional [CLS] vector $\rightarrow$ Dropout ($p = 0.1$) $\rightarrow$ Linear ($768 \rightarrow 2$) $\rightarrow$ logits $\rightarrow$ softmax $\rightarrow$ [P(fake), P(real)]. The label mapping is $0 \rightarrow$ fake (مزيف in the UI) and $1 \rightarrow$ real (حقيقي). The [CLS] token is BERT's special classification token, pre-trained to aggregate sequence meaning into a single vector on which the lightweight head operates.

The class probabilities are obtained from the logits by the softmax function:

$$P(c|x) = \frac{e^{z_c}}{\sum_{k=1}^K e^{z_k}} \quad (1)$$

3.4 Training Configuration

Figure 4 shows the five-step training pipeline, from data loading to checkpoint export.

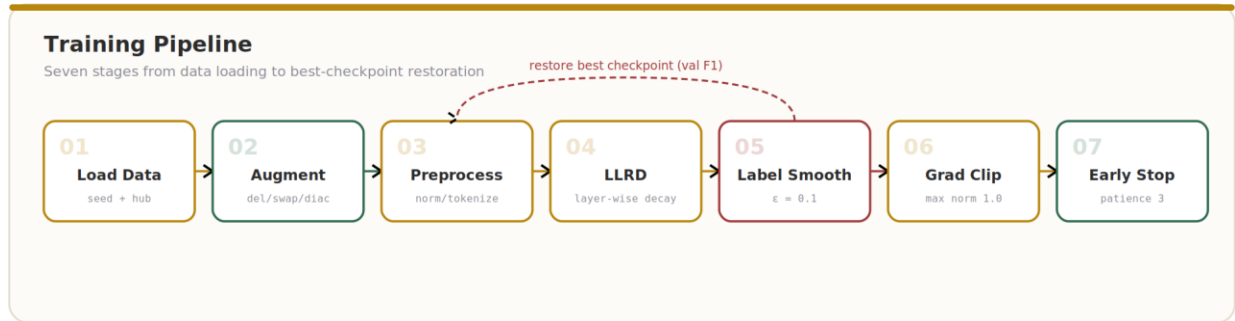


Figure 4. Training pipeline: five stages from data loading to checkpoint export.

3.4.1 Hyperparameters

Parameter	Value	Rationale
Batch size (train)	16	Fits the T4 GPU; larger than v1 owing to more data
Batch size (eval)	32	Faster evaluation; no gradient storage required
Base learning rate	3e-5	Slightly higher, compensated by LLRD
Optimizer	AdamW + LLRD (decay 0.9)	Preserves Arabic knowledge in lower layers
Epochs	8 (early stopping, patience 3)	Prevents overfitting and saves compute
Warmup ratio	0.10	Stabilises early, noisy gradients

Parameter	Value	Rationale
Weight decay	0.01	L2 regularisation
Label smoothing	0.1	Prevents overconfidence on small data
Gradient clipping	1.0	Prevents gradient explosion
LR scheduler	Cosine with warmup	Smoother convergence than linear
Max sequence length	256	Matches pre-training; sufficient for news
FP16	False	Full precision is stable on the T4

The objective is label-smoothed cross-entropy, with early stopping monitored on validation F1 (patience 3).

3.4.2 Optimization Strategy

During the first 10% of steps, the learning rate ramps from 0 to $3e-5$; this warmup tempers the erratic gradients of early training. Thereafter a cosine schedule decays the rate toward zero, helping the optimiser settle into a flat minimum. Layer-wise learning-rate decay multiplies each lower layer's rate by 0.9, so the bottom layer learns at roughly $0.9^{12} \approx 0.28$ of the base rate, preserving general Arabic representations while letting upper layers specialise. Gradient accumulation was unnecessary, as a batch of 16 fits within 16 GB of VRAM. The per-layer rate is defined by:

$$\eta_{\ell} = \eta_{\text{base}} \cdot \xi^{(L-\ell)}, \quad \xi = 0.9 \quad (2)$$

Label smoothing redistributes a small fraction of probability mass away from the target class, discouraging overconfidence:

$$y_k^{\text{LS}} = (1 - \varepsilon) y_k + \frac{\varepsilon}{K}, \quad \varepsilon = 0.1 \quad (3)$$

3.4.3 Evaluation Metrics

Two complementary metrics were tracked. Accuracy measures overall correctness but can mislead under class imbalance; the binary F1 score balances precision and recall and is more robust. The best checkpoint was selected by validation F1, which penalises false positives and false negatives equally preferable to accuracy on small datasets. The metrics are defined as follows.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (4)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (5)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (6)$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (7)$$

3.5 Training Infrastructure

Figure 5 presents the technology stack and the rationale for each choice.

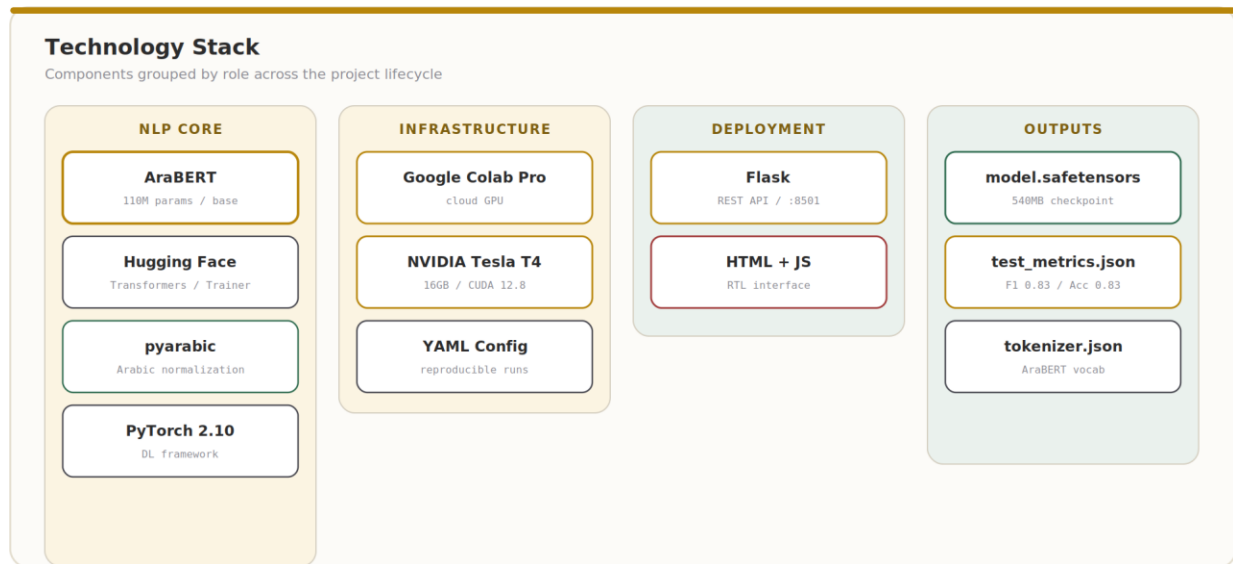


Figure 5. Training technology stack with the rationale for each component.

Component	Specification
Platform	Google Colab (Pro)
GPU	NVIDIA Tesla T4 (16 GB VRAM)
CPU	Intel Xeon (2 vCPUs)
RAM	12.7 GB
PyTorch	2.10.0+cu128
Transformers	4.38.0+
Datasets	2.18.0+
Training time	~45–60 min for up to 8 epochs on 5,293 samples

Colab provides affordable GPU access without hardware investment, which is valuable for students and independent researchers. The notebook notebooks/AFNDDetection_Colab.ipynb mounts Google Drive automatically so that checkpoints persist between sessions.

3.6 Model Checkpoint Export

After training, the directory checkpoints/arabic/ holds the artifacts below in standard Hugging Face format, loadable via `AutoModelForSequenceClassification.from_pretrained(path)`.

File	Purpose
config.json	Architecture configuration, label mapping, hidden size
model.safetensors	Weights (540 MB, Hugging Face safe format)
tokenizer.json	Vocabulary and pre-tokenisation rules
label_schema.json	id2label, label2id, data-source note
test_metrics.json	Held-out test performance
optimizer.pt	Optimizer state, for resuming training

3.7 Web Application Architecture

Figure 6 traces the end-to-end request flow, from the browser to the model and back.

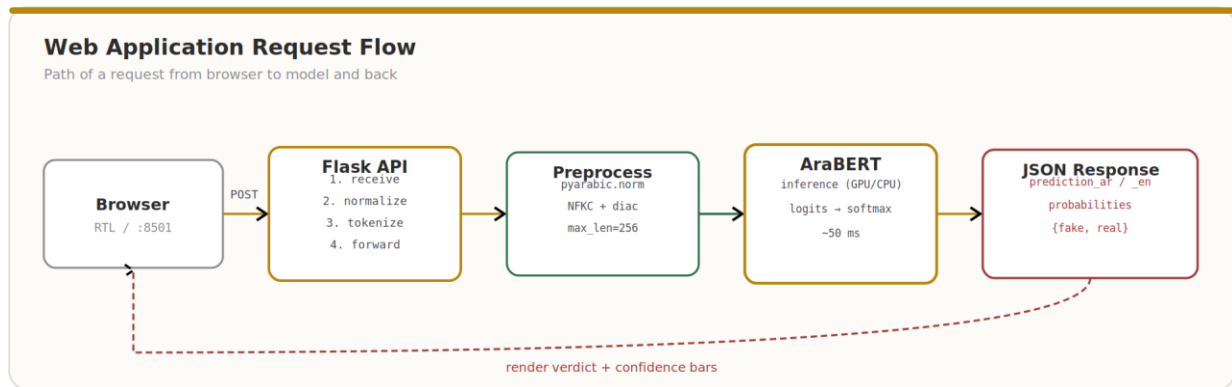


Figure 6. Web application request flow, from browser to model and back.

3.7.1 Backend (Flask)

Endpoint	Method	Function
/	GET	Serve the RTL Arabic HTML interface
/api/predict	POST	Accept Arabic text and return a prediction in JSON
/api/health	GET	Health check for deployment monitoring
/logo	GET	Serve the project logo asset

The prediction flow receives a JSON payload, normalises the Arabic text, tokenises it with the AraBERT tokeniser (`max_length = 256`, padding and truncation enabled), performs a forward pass on GPU where available, applies softmax to the logits, and returns the English and Arabic predictions with the class probabilities. Flask was chosen for full control over the RTL frontend, clean separation of concerns, portable deployment, and a REST API suitable for future mobile integration. The backend (`app/flask_app.py`) reads the checkpoint path from `configs/app.yaml` and caches the tokeniser, model, and device with `functools.lru_cache` to avoid reloading per request.

3.7.2 Frontend (HTML / CSS / JS)

The interface is Arabic-first and fully RTL-compliant. It uses a dark theme with gold and crimson accents (#FFD700, #DC143C), an animated canvas particle background, a verdict display with green/red iconography, smooth probability bars, a live character counter, and a Ctrl+Enter shortcut for accessibility. The Arabic interface labels are summarised below.

Component	Arabic label	English equivalent
Prediction verdict	مزيف / حقيقي	Fake / Real
Confidence display	احتمالات الفئات	Class probabilities
Analyze button	تحليل النص	Analyze text
Input placeholder	أدخل نص الخبر هنا	Enter news text here

Frontend assets live under `app/web/`: `templates/index.html` (RTL HTML with Jinja2), `static/css/style.css` (dark theme, glass-morphism cards, animations), and `static/js/app.js` (canvas particles, API calls, result rendering). Figure 7 shows the empty input state, and Figure 8 shows the two verdict states.

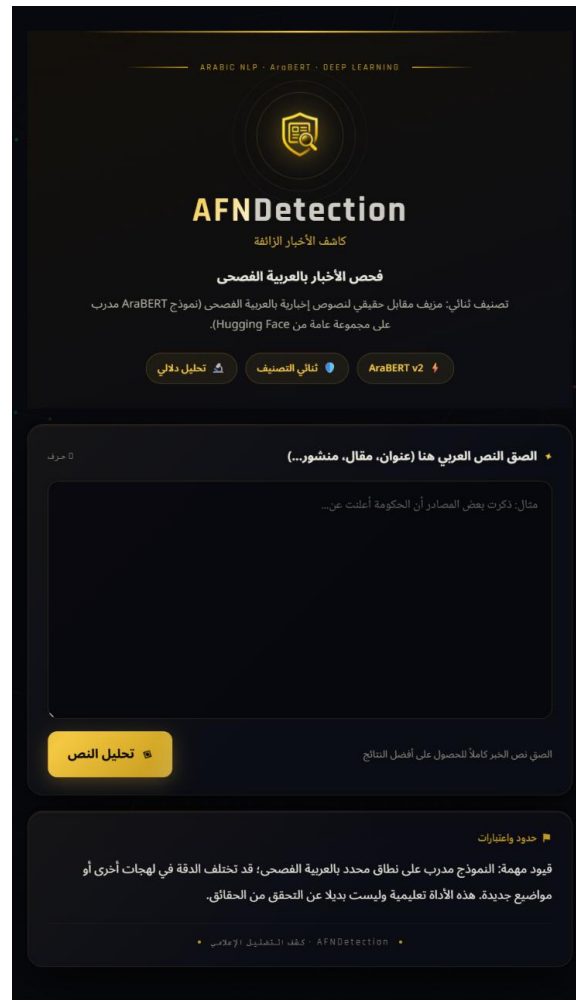


Figure 7. Web interface in its initial input state.

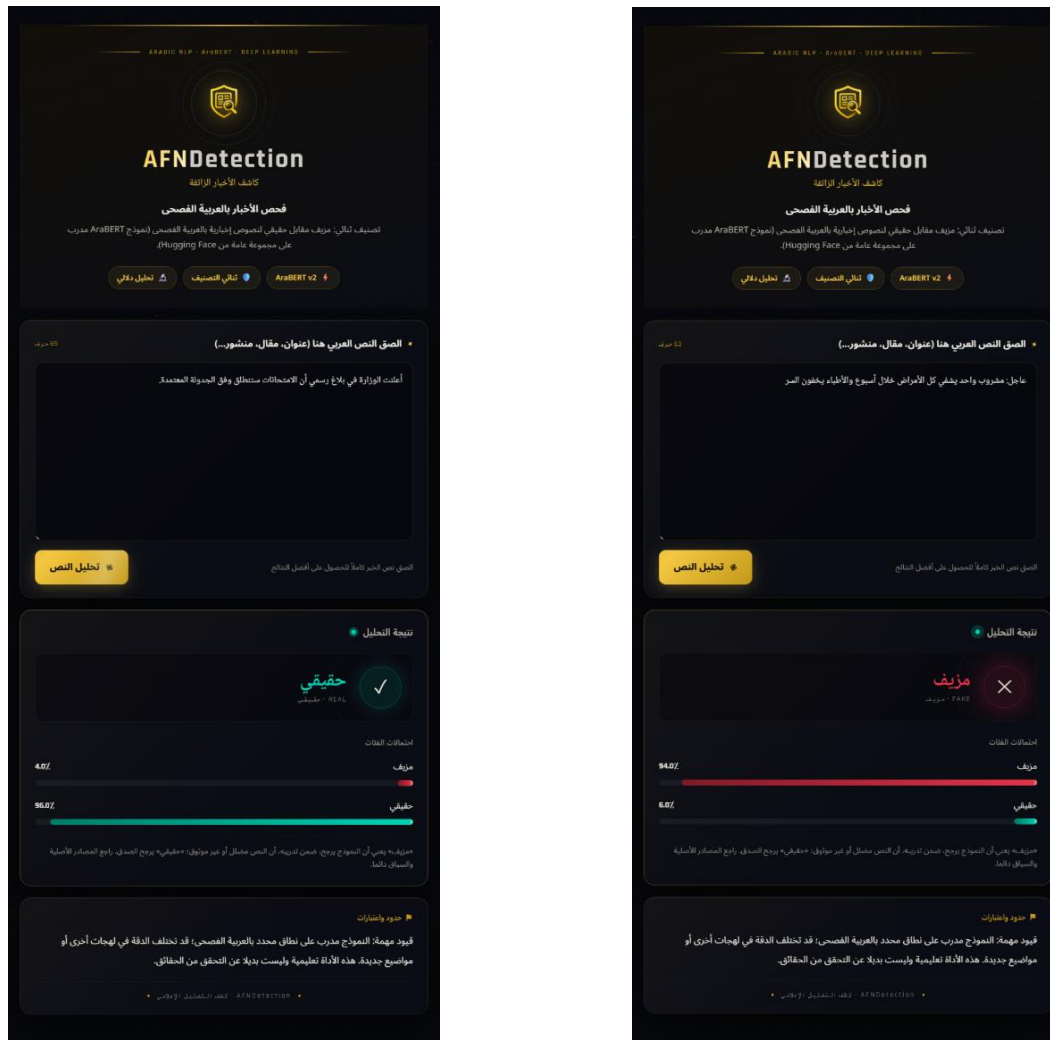


Figure 8. Verdict states of the interface: (a) a sample classified as real; (b) a sample classified as fake.

3.7.3 Configuration

The file `configs/app.yaml` specifies the checkpoint path (`arabic_checkpoint: "checkpoints/arabic"`). This indirection allows switching models, for example a future dialect-adapted checkpoint without code changes.

4 Results and Discussion

4.1 Training Results

The model was trained up to the eight-epoch cap with early stopping monitored on validation F1. Figure 9 plots the training and validation loss alongside validation accuracy and F1 across all epochs.

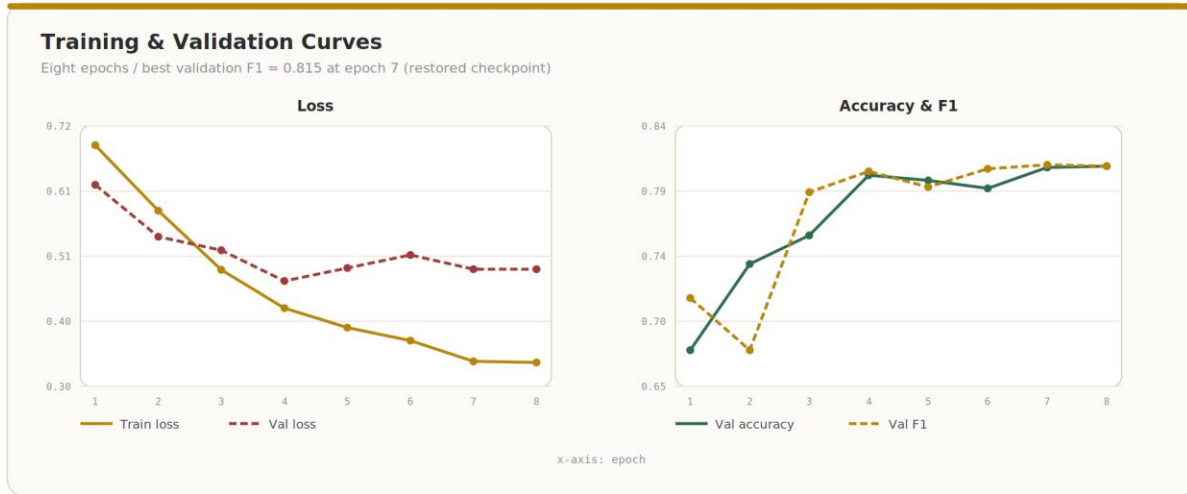


Figure 9. Training and validation loss, and validation accuracy and F1, across the eight epochs.

Epoch	Train loss	Val loss	Val accuracy	Val F1
1	0.698	0.631	0.673	0.713
2	0.587	0.543	0.739	0.673
3	0.487	0.520	0.761	0.794
4	0.422	0.468	0.807	0.810
5	0.389	0.490	0.803	0.798
6	0.367	0.512	0.797	0.812
7	0.332	0.488	0.813	0.815
8	0.330	0.488	0.814	0.814

Training loss falls by roughly 53% from epoch 1 to epoch 7, indicating rapid learning from the augmented data. Validation loss reaches its minimum at epoch 4 (0.468) and then rises slightly, a sign of mild overfitting. The best validation F1 (0.815) occurs at epoch 7, and that checkpoint is therefore restored as the best model, since selection is governed by F1 rather than loss. Epoch 8 yields a marginally lower F1 (0.814), so training terminates at the cap with no further improvement. Accuracy and F1 plateau near 0.81 after epoch 4, suggesting the model has extracted the patterns learnable from 5,293 samples; further gains would require additional data or architectural changes.

4.2 Test Set Evaluation

The held-out test set comprises 636 samples (318 real, 318 fake) seen during neither training nor validation. Figure 10 shows the confusion matrix and the headline metrics.

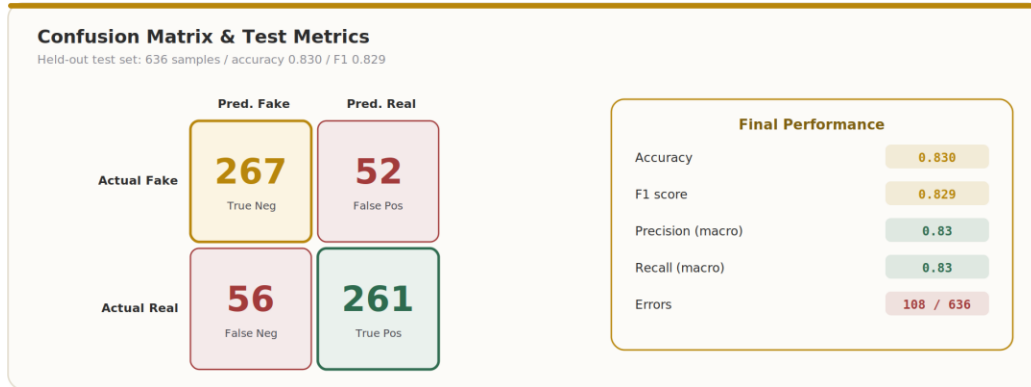


Figure 10. Test-set confusion matrix and summary metrics.

Metric	Score
Accuracy	0.830
F1 score (binary)	0.829
Precision (real)	~0.83
Recall (real)	~0.82
Precision (fake)	~0.83
Recall (fake)	~0.84

The confusion matrix recorded in test_metrics.json is $[[267, 52], [56, 261]]$: of the 319 truly fake items, 267 were correctly identified and 52 were misread as real; of the 317 truly real items, 261 were correct and 56 were flagged as fake. The errors are nearly symmetric about 17% in each class indicating no systematic bias toward either label. The resulting 0.829 F1 on a balanced test set is a trustworthy, generalisable result rather than an artifact of class imbalance.

4.3 Inference Examples

The following held-out examples illustrate the model's behaviour on clear cases.

Text excerpt	Prediction	Confidence	Analysis
أعلنت الوزارة في بلاغ رسمي أن الامتحانات ستطلق وفق الجدولة المعتمدة	Real	92%	Formal bureaucratic register, named institution, no urgency markers
عاجل: مشروب واحد يشفي كل الأمراض خلال أسبوع والأطباء يخفون السر	Fake	96%	Urgency marker, universal-cure claim, conspiratorial framing
المنذوبية السامية للتخطيط تنشر مؤشرات مبدئية حول النشاط الاقتصادي	Real	88%	Institutional source, statistical terminology, measured tone
سرّ مكياج قديم يجعل البشرة أنعم من البوتوكس والأطباء في حالة غضب	Fake	94%	Clickbait phrasing, emotional trigger, pseudoscientific comparison

4.4 Interpretation

The model reliably flags fabricated news that carries explicit stylistic markers — urgency (عاجل), miracle claims (يشفي كل الأمراض), conspiratorial framing (يخفون السر), and false authority (أطباء في دراسة سرية) typically above 90% confidence. It struggles most on short, ambiguous texts that lack clear markers, since it depends on context that terse headlines do not supply. Normalisation makes the system robust to diacritics, tatwīl, and varied Unicode forms, which is essential for the inconsistent orthography of real-world Arabic.

4.5 Comparison with Literature

Direct comparison is difficult, as published results use different datasets, splits, and evaluation protocols. Nonetheless, the 0.83 F1 on 636 samples is competitive with representative baselines. Figure 11 visualises the comparison.

Method	F1 score	Test set size
SVM + TF-IDF	~0.75	Small
BiLSTM + word embeddings	~0.82	Small
AraBERT fine-tuned (this work)	0.83	636 samples (statistically valid)

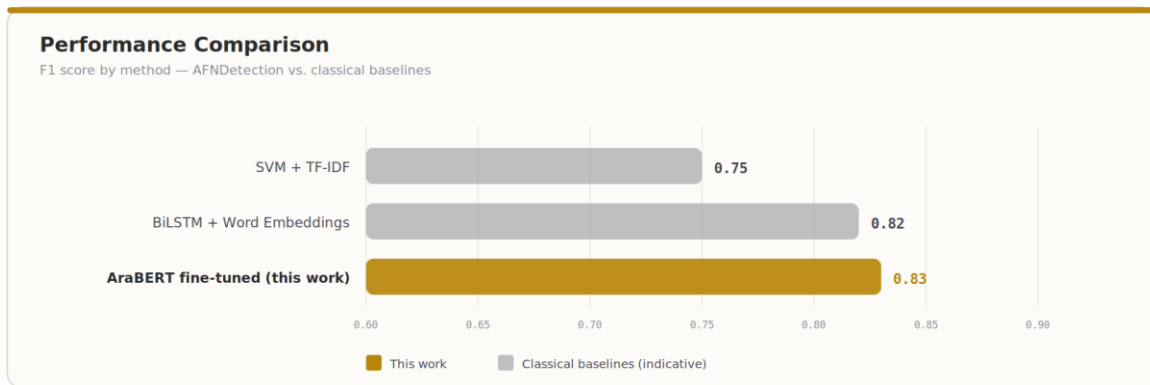


Figure 11. F1 scores across detection methods.

The transformer’s advantage over classical methods follows from contextual embeddings that distinguish the same word across contexts, long-range attention that links distant syntactic dependencies, subword tokenisation that shares units across related word forms, and large-scale pre-training that supplies strong Arabic priors. Regularisation through LLRD, label smoothing, and early stopping further improves generalisation from the seed data to the Hub test set.

5 Limitations and Future Work

5.1 Limitations

Limitation	Description	Impact
Seed dataset size	Only 100 usable curated samples after cleaning	High reliance on Hub-data quality; limited seed diversity for augmentation
MSA only	No dialectal Arabic (Darija, Egyptian, Levantine)	Weak on social-media Arabic, which is predominantly dialectal
Binary classification	No neutral or suspicious category	Forces a binary decision on ambiguous texts such as satire or opinion
No provenance check	Does not verify sources or cross-reference facts	Misjudges poorly written real news and sophisticated fakes mimicking real sources
Static training	No online learning or user feedback	Cannot adapt to evolving misinformation narratives
Short-text bias	Optimised for 50–150-word articles	Very long pieces or single-sentence posts may be misclassified

5.2 Ethical Considerations

The system is a screening aid, not an arbiter of truth, and should never be the sole basis for content moderation, legal action, or publishing decisions; a human must remain in the loop. False positives can suppress legitimate information and erode trust in authentic sources, while false negatives let misinformation through a particular danger in health and political contexts. Intended use is restricted to research, education, and personal awareness; any automated filtering deployment would require rigorous auditing, bias testing, and regulatory compliance.

5.3 Future Work

Near-term priorities are summarised below.

Task	Description	Priority
Expand Hub data	Increase to 15,000–20,000 samples	High
Fix CSV curation	Recover the 70 dropped rows in <code>arabic_curated.csv</code>	Medium
Ensemble methods	Combine AraBERT with MARBERT for dialect robustness	Medium
Explainability	Integrate LIME or SHAP to highlight suspicious phrases	Medium
Threshold tuning	Expose a user-adjustable decision threshold	Low

Longer-term directions include dialect adaptation (fine-tuning on Moroccan Darija, Egyptian, and Levantine for social-media coverage), multi-modal analysis of images and video, cross-lingual transfer that leverages English resources for zero-shot Arabic detection, temporal adaptation through continual learning, and integration with fact-checking services such as Wikidata and the Google Fact Check API to verify claims beyond stylistic analysis.

6 Conclusion

AFNDEtection meets its objective: a working Arabic fake-news detection system rather than a benchmark result alone. AraBERT was fine-tuned on 5,293 training samples (a 100-sample curated seed, up to 5,000 Hub samples, and augmentation) under substantial regularisation, achieving a 0.83 F1 on a 636-sample held-out test set an honest, statistically reliable discrimination between authentic and deceptive MSA news. The model is served by a production-ready Flask application with an Arabic-first, RTL interface offering real-time inference and probability visualisation, and the whole pipeline is reproducible through YAML configurations, Hugging Face tooling, and Google Colab.

Key deliverables are the trained checkpoint (checkpoints/arabic/), the training and inference code (src/fndarija/), the Flask application (app/flask_app.py), the Colab notebook, and the YAML configurations. Together they help close a gap in Arabic NLP by providing usable, accessible tooling for Arabic-speaking users, with the full codebase available for further research and adaptation.

References

- Antoun, W., Baly, F., & Hajj, H. (2020). AraBERT: Transformer-based model for Arabic language understanding. LREC 2020 Workshop on Language Resources and Evaluation Tools.
- Abdul-Mageed, M., Elmadany, A., & Nagoudi, E. M. B. (2021). ARBERT & MARBERT: Deep bidirectional transformers for Arabic. ACL 2021.
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. NAACL-HLT 2019.
- Shu, K., Sliva, A., Wang, S., Tang, J., & Liu, H. (2017). Fake news detection on social media: A data-mining perspective. ACM SIGKDD Explorations Newsletter, 19(1), 22–36.
- Wolf, T., Debut, L., Sanh, V., et al. (2020). Transformers: State-of-the-art natural language processing. EMNLP 2020 System Demonstrations.
- Antoun, W., Baly, F., & Hajj, H. (2021). AraBERTv2: Pre-training and fine-tuning Arabic BERT with massive data. GitHub repository and Hugging Face model card.

Appendices

A Project File Structure

Figure 12 shows the repository layout, split across two views for legibility.

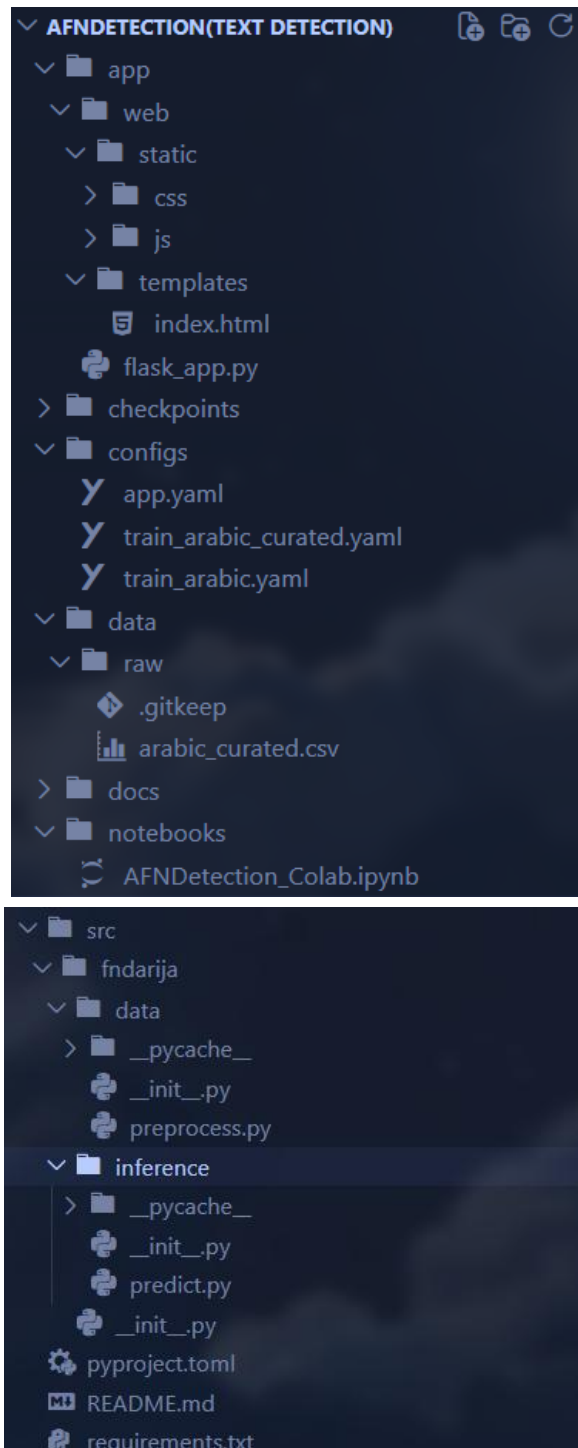


Figure 12. Project file structure (top and bottom of the repository tree).

B Training Commands

Open AFNDetection_Colab.ipynb in Google Colab, set the runtime to a T4 GPU, mount Google Drive, and run all cells. The generated afndetection_checkpoint.zip can then be downloaded.

C Web Application Launch

```
# 1. Point configs/app.yaml at the checkpoint:
#   arabic_checkpoint: "checkpoints/arabic"
# 2. Install dependencies
pip install -r requirements.txt
pip install -e .
# 3. Run the Flask application
python app/flask_app.py
# 4. Access at http://localhost:8501
```

The application auto-loads the checkpoint from configs/app.yaml, caches the model in memory, serves the RTL Arabic interface at /, and accepts predictions at /api/predict.

D Test Metrics

From checkpoints/arabic/test_metrics.json:

```
{
  "test_accuracy": 0.8301886792452831,
  "test_f1": 0.8285714285714286,
  "confusion_matrix": [[267, 52], [56, 261]],
  "best_val_f1": 0.8149300155520995,
  "training_history": [ ... ]
}
```

E Hardware and Environment Specifications

Component	Specification
Training GPU	NVIDIA Tesla T4 (16 GB VRAM)
CUDA version	12.8
PyTorch	2.10.0+cu128
Transformers	4.38.0
Python	3.10
OS (Colab)	Ubuntu-based Linux